Agent Run Report — Healthcare Q&A Agent

A short walkthrough of what I built, how it thinks, and how it behaves when you actually run it. The goal was a small, honest agent that answers health questions by looking things up and summarizing what it finds — not a giant framework, and nothing that needs servers to run.

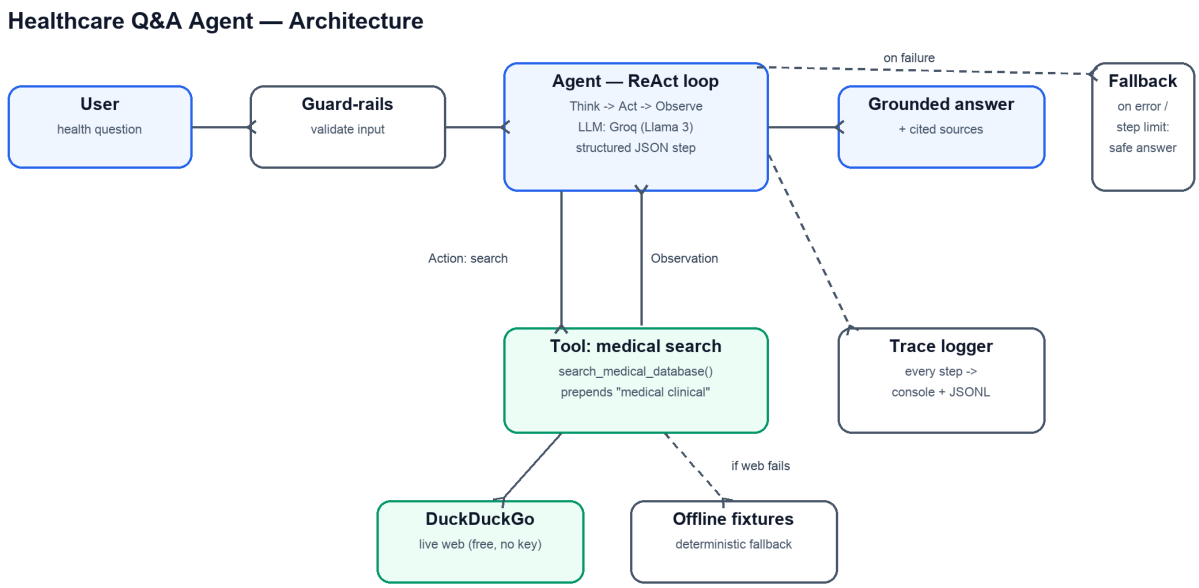
Live demo: https://healthcare-agent-fo48sgeutc25uy2ogrqpk6.streamlit.app/

1. What it does, in one line

You ask a health question, the agent searches a medical knowledge base, reads the results, and writes back a short answer with the sources it used.

I picked the **healthcare Q&A** domain because it shows the whole agent story clearly: it has to decide *what* to look up, actually call a tool, read the results, and then ground its answer in them.

2. Architecture



The flow is a classic **ReAct loop** (Reason → Act → Observe), which mirrors how a person researches something:

- **Guard-rails** — sanity-check the question before spending any tokens.
- **Agent (the LLM)** — each turn returns one small JSON step: a *thought* plus an *action* (either "search" or "final answer").
- **Tool: medical search** — search_medical_database() uses DuckDuckGo (free, no key) as a stand-in for PubMed, steering queries toward medical sources.
- **Observation** — the top results (title, snippet, link) go back to the agent.

- The loop repeats until the agent is confident, then it writes a **grounded answer with cited sources**.

Two supporting pieces make it robust: a **trace logger** that records every step to the console and a JSONL file, and a **fallback** that returns a safe answer if the LLM errors or the step budget runs out (and the search tool falls back to bundled fixtures if the web is unavailable).

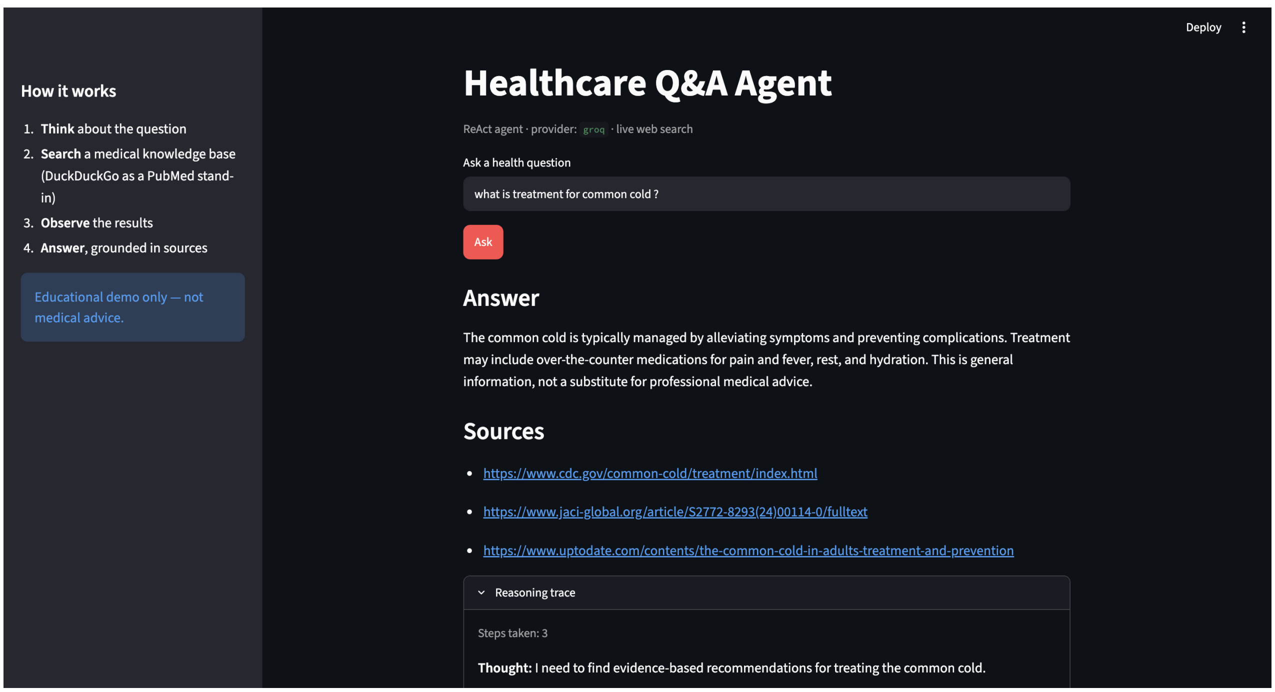
Why these choices

- **Free + zero infra:** Groq's free tier (Llama 3) for the LLM, DuckDuckGo for search.
- **Custom loop, not a framework:** the deliverable is the reasoning trace, so I kept the loop small (~40 lines) and readable instead of hiding it in LangChain.
- **Structured JSON every step:** validated with Pydantic for clean parsing.

3. The app in action

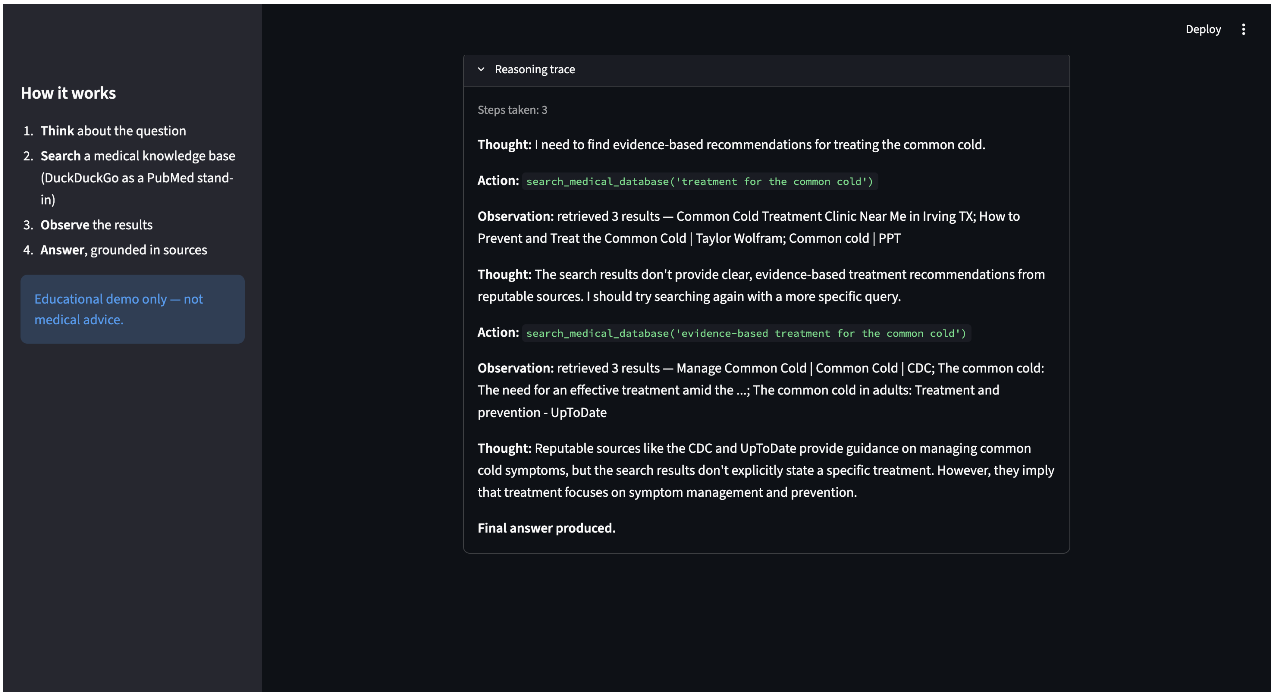
A simple Streamlit UI wraps the same agent. These are real runs (Groq + live DuckDuckGo search).

Example A — "What is the treatment for the common cold?"

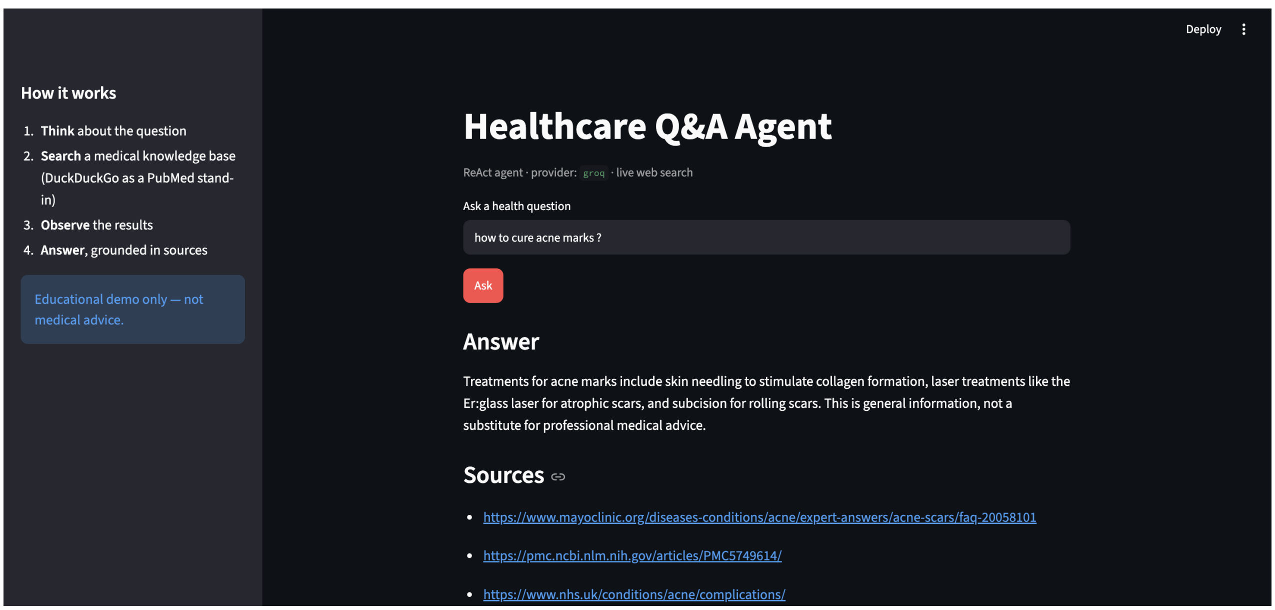


The agent answers and lists the sources it actually used.

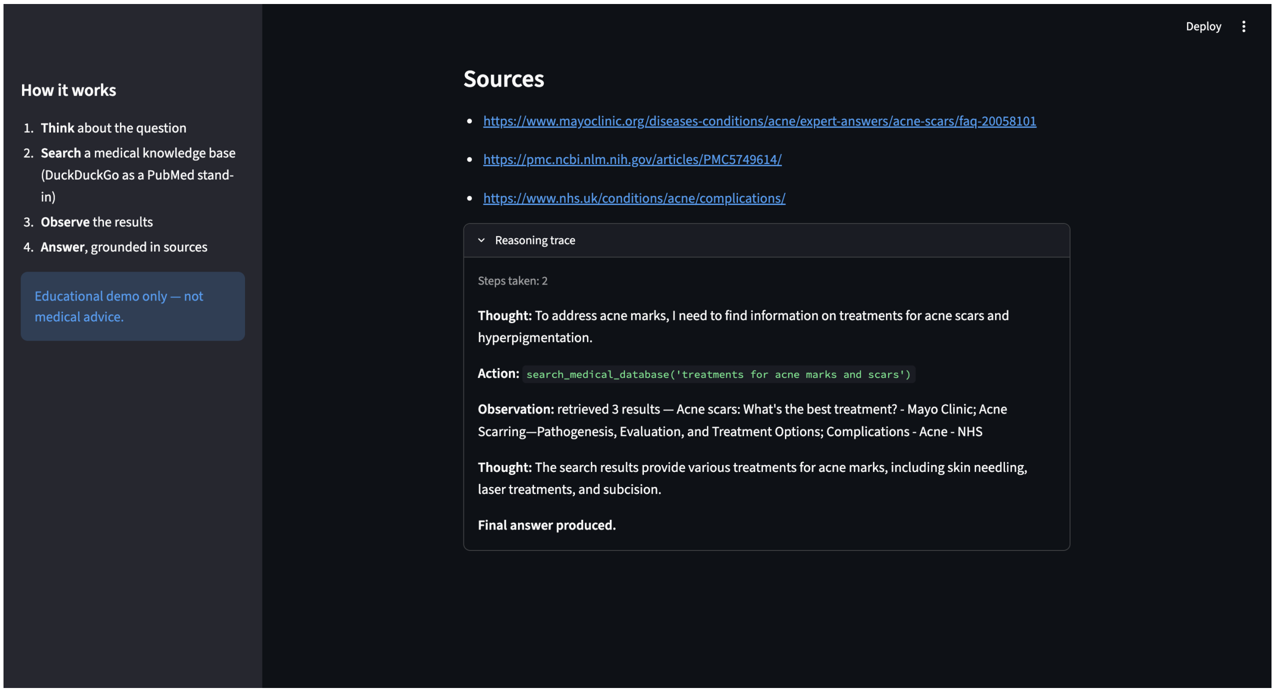
The nice part is the trace below: the agent's first search returned weak results, so it noticed and searched again with a better query before answering — genuine reasoning, not a single canned lookup:



Example B — "How to cure acne marks?"



A different topic, same clean behavior, with Mayo/NIH/NHS sources.



4. A full reasoning trace (text)

The same common-cold run captured from the terminal — the exact reasoning chain, tool calls, and final output:

```
[QUERY] what is treatment for common cold?

[THOUGHT]  I need to find evidence-based recommendations for treating the common cold.
[ACTION]   search_medical_database('treatment for the common cold')
[OBSERVE]  3 results - weak / non-authoritative sources

[THOUGHT]  These results aren't from reputable sources. I should search again
           with a more specific query.
[ACTION]   search_medical_database('evidence-based treatment for the common cold')
[OBSERVE]  3 results - CDC; review article; UpToDate

[THOUGHT]  Reputable sources (CDC, UpToDate) point to symptom management and prevention.
[ANSWER]   The common cold is typically managed by alleviating symptoms - OTC
           pain/fever medicine, rest, and hydration. This is general information,
           not a substitute for professional medical advice.
[SOURCES]  cdc.gov/common-cold/treatment, jaci-global.org/..., uptodate.com/...
```

5. Evaluation

5 scenarios in tests/scenarios.json, run by evaluate.py, checking each answer against simple expectations. It runs fully offline (mock LLM + fixtures), so results are deterministic.

#	Scenario	What it checks	Result
1	Type 2 diabetes treatment	mentions metformin, cites sources	PASS

2	First-line hypertension therapy	mentions a first-line drug class	PASS
3	Asthma long-term treatment	mentions corticosteroid	PASS
4	Empty query	guard-rail rejects the input	PASS
5	Unknown topic	falls back to general trusted sources	PASS

Score: 5/5.

6. Design decisions & trade-offs

- **DuckDuckGo instead of PubMed.** Same engineering for free; trade-off is lower medical authority — acceptable given the assignment values engineering over accuracy.
- **Custom ReAct over LangChain.** Smaller, clearer, no dependency churn; trade-off is no built-in memory, which this scope doesn't need.
- **Mock LLM + offline fixtures.** The project and its evaluation run with no key and no network; switching to Groq is one env-var change.
- **Degrade gracefully.** LLM error → retry then fallback; search failure → fixtures; too many steps → safe answer. It never just crashes on the user.

7. How to run it

Try it live (no setup): https://healthcare-agent-fo48sgeutc25uy2ogrqpk6.streamlit.app/

```
pip3 install -r requirements.txt
cp .env.example .env          # add your free Groq key (console.groq.com/keys)

# Ask a question (Groq + live web search):
python3 -m src.agent --domain healthcare --query "Treatment options for Type 2 diabetes?"

# Or launch the UI:
python3 -m streamlit run app.py
```

Educational demo only — not a substitute for professional medical advice.